UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Software

Informe de Desarrollo del Examen Parcial

Programación Concurrente y Paralela

Curso: Programación Concurrente y Paralela

Docente: Ing. Pablo E. López V.

Semestre académico: 2026-II

Fecha: 15 de octubre de 2026

Integrante(s):

Castilla Huanca Marco

Basualdo Ale Marcos

Aldana Chavez David

De La Cruz Meza Angel

Poma Gutierrez Gabriel

1. Introducción

El trabajo desarrolla dos aplicaciones Java para resolver el examen parcial de Programación Concurrente y Paralela. La primera organiza ocho operaciones sobre archivos mediante dependencias; la segunda distribuye transacciones bancarias entre trabajadores conectados por TCP. Ambas incluyen una ejecución serial de referencia y verificaciones de equivalencia.

En el paralelismo orientado a tareas se ejecutan operaciones diferentes cuando sus entradas están disponibles. En el paralelismo orientado a datos se aplica el mismo procesamiento a particiones distintas del dataset. Esta diferencia determina los mecanismos utilizados: CompletableFuture y un conjunto de hilos para la primera pregunta; un coordinador, sockets y agregación de resultados parciales para la segunda.

La sincronización evita consumir archivos incompletos y la lectura por bloques limita la memoria utilizada. En el clúster, los pesos de capacidad y el almacenamiento disponible determinan las cuotas. La validación confirmó resultados equivalentes, aunque las mediciones no muestran una aceleración estable: la sobrecarga, el disco y las condiciones de ejecución deben considerarse al interpretar el rendimiento.

2. Objetivos

2.1. Objetivo general

Desarrollar aplicaciones Java que implementen paralelismo orientado a tareas y paralelismo orientado a datos, comparando ejecuciones seriales y paralelas mediante operaciones reales sobre archivos y distribución de carga.

2.2. Objetivos específicos

• Aplicar RandomAccessFile, lectura por bloques y seek() sobre archivos físicos.

• Sincronizar las tareas A–H con hilos y CompletableFuture.

• Distribuir archivos entre JVM trabajadoras identificadas por IP y configuración.

• Balancear por capacidad y almacenamiento; consolidar importes en céntimos.

• Verificar SHA-256, tiempos y detección de trabajos incompletos.

3. Desarrollo de la pregunta 1

3.1. Enunciado y análisis del problema

El examen solicita versiones serial y paralela con ocho nodos y dependencias funcionales. El enunciado impreso también plantea búsqueda de patrones y tendencias en datasets multidimensionales (López V., 2026). Las precisiones de implementación recogidas en las indicaciones suministradas priorizan operaciones reales sobre archivos mediante RandomAccessFile. Por ello, el proyecto procesa respaldos de registros y añade una búsqueda del patrón ERROR en el resultado final.

La solución replica contenido cuando "multiplica" un archivo; no implementa multiplicación de matrices. La búsqueda cuenta apariciones de una cadena: no calcula correlaciones ni tendencias estadísticas. Esta delimitación diferencia el procesamiento efectivamente desarrollado de la descripción más amplia del examen.

3.2. Diseño de la solución

Principal.nodo(char) selecciona la operación. Las entradas son entrada/registro1.txt y entrada/registro2.txt. Cada modo produce su propia carpeta salida/serial o salida/paralelo. Los archivos de las tareas se escriben primero con extensión .tmp y se renombran a .dat después del cierre.

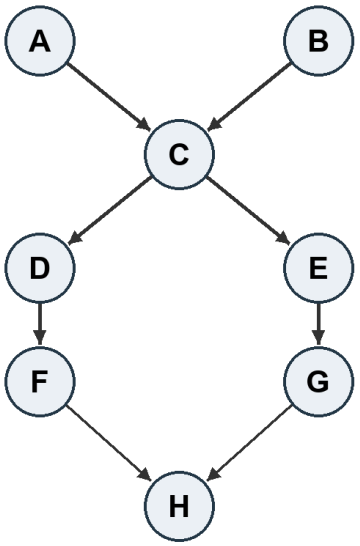
Tabla 1. Operaciones implementadas en los ocho nodos

Nodo	Operación	Entrada	Salida	Dep.
A	Copia por bloques	registro1.txt	A.dat	—
B	Copia por bloques	registro2.txt	B.dat	—
C	Concatenación	A.dat + B.dat	C.dat	A, B
D	Repetición triple	C.dat	D.dat	C
E	Mayúsculas ASCII in situ sobre una copia	C.dat	E.dat	C
F	Extracción del 50 % central	D.dat	F.dat	D
G	Extracción del 50 % central	E.dat	G.dat	E
H	Concatenación y conteo de ERROR	F.dat + G.dat	H.dat	F, G

Fuente: Pregunta1/src/Principal.java y Operaciones.java. Las extracciones se realizan por bytes; pueden cortar una línea de texto.

3.3. Grafo de dependencias funcionales

Figura 1. Dependencias de las tareas implementadas



Fuente: elaboración a partir del diagrama del examen y de Principal.ejecutar(boolean).

A y B no requieren resultados previos y pueden comenzar simultáneamente. C necesita ambas copias completas. Después de C se abren dos ramas: D produce el archivo replicado que consume F, mientras E genera la transformación que consume G. H requiere la terminación de F y G.

No existe una barrera adicional entre D y E: G puede comenzar al terminar E aunque D siga ejecutándose. De igual forma, F solo espera a D. Esta propiedad conserva el grafo del examen y permite aprovechar la independencia de las ramas. La disponibilidad se expresa mediante futuros completados correctamente; un antecesor que falla no habilita sus continuaciones normales.

3.4. Implementación serial

Serial.main delega en Principal.main con el modo serial. Principal.ejecutar(false) invoca nodo(char) en el orden A, B, C, D, E, F, G y H. Aunque el método crea el ejecutor común a ambos modos, la rama serial no le envía tareas: todas las operaciones se ejecutan en el hilo main.

Listado 1. Rama serial de Principal.ejecutar(boolean)

```
if(!paralelo) for(char c='A';c<='H';c++) nodo(c);
```

El orden secuencial respeta todas las dependencias. Cuando comienza C, las llamadas de A y B ya retornaron; cuando comienza H, F y G terminaron. No se necesita una espera activa ni coordinación entre hilos en esta rama.

Operaciones.unir crea o abre el destino en modo rw, elimina contenido previo mediante setLength(0) y copia cada entrada con segmento. D reutiliza unir pasando tres veces C.dat. E llama a transformar, mientras F y G llaman a extraer. H concatena y luego cuenta ERROR usando patrones. La comprobación hash se realiza después del trabajo medido.

Listado 2. Operaciones reales seleccionadas por nodo

```
case 'C' -> Operaciones.unir(temporal,p('A'),p('B'));
case 'D' -> Operaciones.unir(temporal,p('C'),p('C'),p('C'));
case 'E' -> Operaciones.transformar(p('C'),temporal);
case 'F' -> Operaciones.extraer(p('D'),temporal);
case 'G' -> Operaciones.extraer(p('E'),temporal);
```

Fuente: Pregunta1/src/Principal.java, método nodo(char).

Al finalizar cada tarea, Files.move reemplaza su salida publicada. La implementación no solicita ATOMIC_MOVE: el control de dependencias y el cierre previo evitan que las tareas de esta ejecución lean una salida aún en escritura. Esto no constituye una garantía de recuperación transaccional ante fallos del sistema operativo.

3.5. Implementación paralela

Principal.ejecutar(true) utiliza Executors.newFixedThreadPool(4). A y B se envían con CompletableFuture.runAsync. allOf(a,b) representa la condición conjunta que habilita C; thenRunAsync expresa las dependencias restantes. Según la API de Java, estas continuaciones normales se ejecutan tras la finalización correcta de sus etapas previas (Oracle, s. f.-b).

Listado 3. Sincronización del grafo en Principal.ejecutar(boolean)

```
var c=CompletableFuture.allOf(a,b)
    .thenRunAsync(()->nodo('C'),pool);
var d=c.thenRunAsync(()->nodo('D'),pool);
var e=c.thenRunAsync(()->nodo('E'),pool);
var f=d.thenRunAsync(()->nodo('F'),pool);
var g=e.thenRunAsync(()->nodo('G'),pool);
CompletableFuture.allOf(f,g)
    .thenRunAsync(()->nodo('H'),pool).join();
```

Fuente: Principal.java. Se ajustaron saltos de línea del fragmento para su presentación; las expresiones corresponden al código original.

join espera el resultado terminal del grafo. Cada tarea usa sus propios descriptores y escribe un archivo distinto, mientras los antecesores publicados se abren para lectura. Por ello no se comparte un puntero RandomAccessFile entre hilos ni se escribe simultáneamente en la misma salida.

El registro usa una lista sincronizada y el método sincronizado anotar. Cada evento indica nodo, INICIO/FIN/ERROR, nombre del hilo, System.nanoTime, duración y fecha UTC. Esta instrumentación permite comprobar precedencias y solapamiento sin inferir concurrencia solo a partir del número de hilos creados.

Los errores de entrada/salida se convierten en CompletionException; la cadena dependiente no continúa normalmente. El bloque finally cierra el ejecutor, espera su terminación y guarda registro.csv. La aplicación informa el error y termina con código 1. No se utilizan Thread.sleep ni pausas para aparentar carga de trabajo.

3.6. Uso de RandomAccessFile

RandomAccessFile permite leer y escribir un archivo con un puntero de posición que puede modificarse mediante seek(long) (Oracle, s. f.-c). En Operaciones.segmento, seek(inicio) ubica el primer byte de interés y un buffer de 65 536 bytes limita cada lectura. El destino recibe exactamente los bytes efectivamente leídos.

Listado 4. Copia de un segmento físico por bloques

```
    try (RandomAccessFile f = new RandomAccessFile(origen.toFile(), "r")) {
        f.seek(inicio);
        byte[] b = new byte[BLOQUE];
        while (cantidad > 0) {
            int n = f.read(b, 0, (int)Math.min(b.length, cantidad));
            if (n < 0) throw new EOFException("Segmento incompleto: " + origen);
            destino.write(b, 0, n);
            cantidad -= n;
        }
    }
}
```

Fuente: Operaciones.java, segmento(Path, RandomAccessFile, long, long).

Operaciones.extraer calcula inicio = tamaño/4 y cantidad = tamaño/2, con división entera. El método no necesita mantener el archivo completo en RAM. Si el origen termina antes de copiar la cantidad prevista, se lanza EOFException y no se publica el resultado como una tarea exitosa.

En transformar, primero se copia C.dat. Luego se lee cada bloque, se sustituyen únicamente bytes ASCII entre a y z, se retrocede con seek(posicion), se escribe el bloque modificado y se avanza al siguiente. No se realiza una conversión general de mayúsculas Unicode; esta operación es adecuada para las entradas ASCII generadas.

La búsqueda patrones conserva el estado del algoritmo KMP entre lecturas, de modo que ERROR puede comenzar al final de un bloque y terminar en el siguiente. PruebaPatrones verifica ese límite. Los arreglos son buffers de transferencia y estado del patrón, no una representación en memoria del dataset. Los archivos son físicos, aunque el sistema operativo puede atender parte de la E/S desde su caché; el código no fuerza cada escritura al dispositivo con sync().

3.7. Resultados de ejecución

La revalidación se ejecutó sobre los fuentes extraídos del ZIP, con JDK 25.0.1 y compilación --release 17. Principal generar 16777216 creó dos entradas de 16 MiB. La ejecución comparar verificó igualdad SHA-256 en A.dat–H.dat. Los ocho nodos terminaron en ambos modos.

Tabla 2. Resultados de la revalidación de la pregunta 1

Indicador	Serial	Paralelo
Tiempo de ejecución	712,352 ms	3 976,087 ms
Tareas ejecutadas	8, hilo main	8, ejecutor de 4 hilos
Archivos de datos	A.dat–H.dat	A.dat–H.dat
Tamaño de H.dat	67 108 864 bytes	67 108 864 bytes
Resultado final	SHA-256 equivalente	SHA-256 equivalente

Fuente: informe_trabajo/revalidacion/p1_comparar.log y salidas de esa ejecución. Además se generan registro.csv y EXITO.txt por modo.

Speedup = 712,352 / 3 976,087 = 0,179. En esta ejecución, el procesamiento paralelo tardó aproximadamente 5,58 veces el tiempo serial. Un registro anterior conservado en el ZIP presenta 2 875,833 ms seriales y 2 110,523 ms paralelos, con speedup 1,363. Se trata de observaciones individuales, no de promedios ni de un experimento con cachés controladas.

Listado 5. Extracto textual del registro de comparación incluido en el ZIP

```
H,FIN,pool-2-thread-3,124519155468600,298703800,2026-10-16T00:39:45.217694400Z
Tiempo serial: 2875.833 ms
Tiempo paralelo: 2110.523 ms
Speedup: 1.363
Validacion SHA-256 A-H: OK
```

3.8. Análisis de la pregunta 1

Los registros muestran ejecución superpuesta de A/B y D/E, y todas las precedencias del grafo se verificaron. Sin embargo, el paralelismo comparte almacenamiento y genera tráfico de E/S adicional concurrente. La creación de hilos, el registro y la búsqueda también influyen. La variación observada es compatible con efectos de caché, contención o actividad del equipo, pero estos factores no se midieron por separado. La evidencia demuestra sincronización y equivalencia; no una aceleración generalizable.

4. Desarrollo de la pregunta 2

4.1. Enunciado y análisis del problema

La segunda pregunta exige repartir un repositorio en disco entre N computadoras conectadas en red, con capacidad de procesamiento y almacenamiento heterogéneos. La solución implementa un maestro que distribuye archivos y trabajadores que ejecutan la misma agregación sobre particiones disjuntas.

ClusterComputing_1.java identifica la IP local y consulta una configuración para seleccionar trabajo; Configuracion.txt asocia nombres, IP y códigos territoriales (López V., s. f.-a, s. f.-b). El proyecto conserva la identificación configurable y adopta un dominio diferente: transacciones bancarias sintéticas. Sustituye el acceso a rutas de red codificadas por transferencias TCP efectivas.

4.2. Caso de estudio

Cada registro representa un depósito o retiro de una cuenta. El objetivo es obtener cantidad de operaciones, total depositado, total retirado y variación neta por cuenta, junto con un TOTAL global. La fecha y la sucursal describen la transacción, pero no son dimensiones de agrupación en la versión desarrollada.

Tabla 3. Estructura del dataset bancario

Campo	Contenido y validación
id_transaccion	Entero positivo, único y estrictamente creciente
id_cuenta	Identificador CTA seguido de 5 a 9 dígitos
fecha	Fecha aceptada por LocalDate.parse
tipo	DEPOSITO o RETIRO
monto	Decimal positivo convertible exactamente a céntimos
sucursal	Cadena en mayúsculas A–Z

Generador.main escribe línea por línea con semilla 20261015. Permite configurar registros y cuentas; la prueba principal empleó 100 000 transacciones y 1 000 cuentas. El ZIP incluye el generador y una muestra de 100 filas.

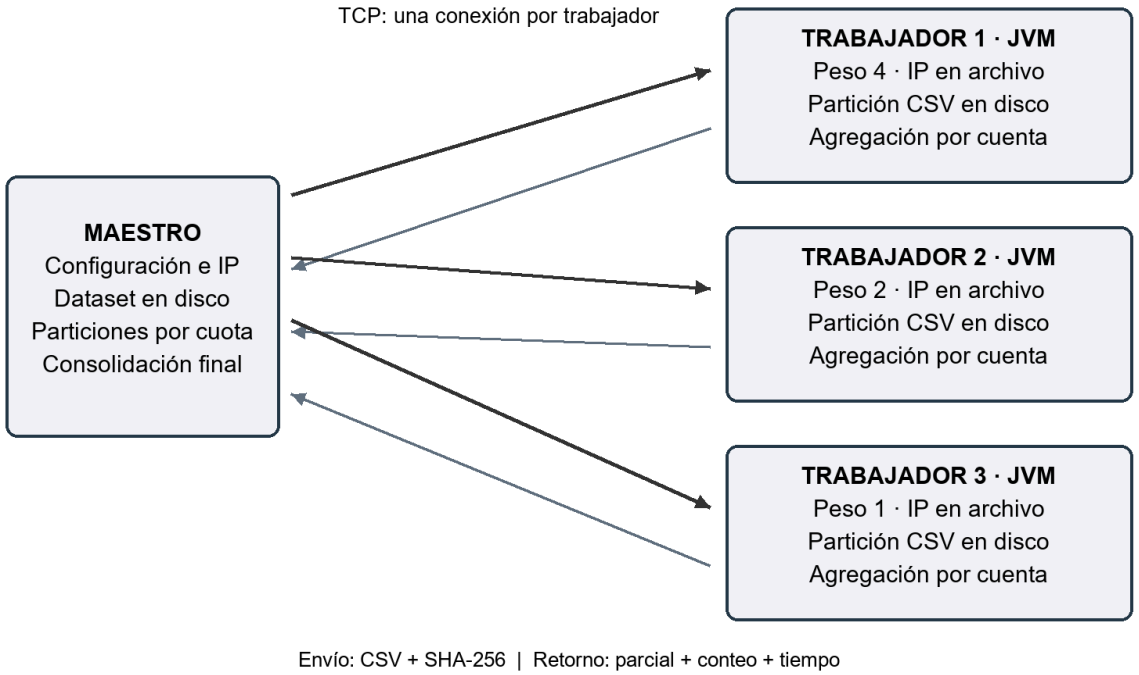
Listado 6. Filas reales de dataset/muestra.csv

```
id_transaccion,id_cuenta,fecha,tipo,monto,sucursal
1,CTA00003,2026-10-01,RETIRO,1474.26,AREQUIPA
2,CTA00001,2026-10-02,RETIRO,2899.68,AREQUIPA
3,CTA00001,2026-10-24,RETIRO,1746.00,LIMA
```

Fuente: dataset/muestra.csv, generado con 10 cuentas; la prueba principal usó 1 000 cuentas.

4.3. Arquitectura del sistema distribuido

Figura 2. Maestro y trabajadores conectados mediante TCP



Fuente: Coordinador.java, Trabajador.java y Transferencia.java. En la prueba local, todas las IP son 127.0.0.1 y cada trabajador es un proceso independiente.

Coordinador.distribuir inspecciona el dataset, abre ServerSocket, admite trabajadores configurados, calcula cuotas y escribe una partición CSV por nodo. Un ExecutorService con tantos hilos como conexiones aceptadas gestiona los envíos y retornos simultáneos. El cálculo bancario se realiza en las JVM trabajadoras, no en esos hilos del maestro.

Trabajador.main lee su IP, comprueba la identidad, informa espacio libre y recibe un UUID de trabajo, cuota esperada y archivo. La partición queda guardada en trabajo/nodoN/UUID/particion.csv. Procesamiento.procesar genera el agregado; Resumen.guardar produce parcial.csv. El resultado, conteo y tiempo regresan al maestro.

Transferencia.enviar transmite longitud, SHA-256 y bytes. recibir limita el tamaño, escribe un temporal, comprueba integridad y publica el archivo solo si la transferencia es completa. El maestro valida conteos, combina cuentas y escribe consolidado.csv. El protocolo utiliza DataInputStream y DataOutputStream sobre sockets TCP; la API Socket proporciona la conexión entre extremos de red (Oracle, s. f.-d).

Cada trabajador procesa un trabajo por conexión y termina. No existe un servicio persistente de planificación, descubrimiento automático de equipos ni migración de tareas durante la ejecución.

4.4. Configuración de direcciones IP

Configuracion carga cluster.properties con Properties y construye registros Nodo con ID, IP, peso y límite. La IP del maestro se toma de ese archivo; cada trabajador además lee su archivo nodoN-ip.txt mediante leerIp(Path). En el escenario local, este archivo contiene 127.0.0.1.

Listado 7. Configuración original incluida en el proyecto

```
maestro.ip=127.0.0.1
maestro.puerto=5000
espera.ms=10000
socket.ms=120000
nodos=nodo1,nodo2,nodo3
nodo1.ip=127.0.0.1
nodo1.capacidad=4
nodo1.almacenamiento.mb=64
nodo2.ip=127.0.0.1
nodo2.capacidad=2
nodo2.almacenamiento.mb=32
nodo3.ip=127.0.0.1
nodo3.capacidad=1
nodo3.almacenamiento.mb=16
```

maestro.puerto fija el puerto de escucha. espera.ms limita la etapa de registro de trabajadores y socket.ms limita la inactividad de lectura de cada conexión. Los valores almacenamiento.mb se convierten a bytes usando 1 048 576; por ello representan MiB, aunque la propiedad use el sufijo mb.

Configuracion.ip valida IPv4 numéricas y octetos. local comprueba que la dirección corresponda a una interfaz del equipo, salvo loopback. El trabajador compara su archivo de IP con la configuración; el maestro comprueba ID, IP declarada, dirección remota y duplicados. Los IDs distinguen a varios procesos en 127.0.0.1. Estas comprobaciones no sustituyen autenticación criptográfica.

El modo Coordinador ip archivo [IPv4] guarda una dirección validada. Si se omite IPv4, utiliza InetAddress.getLocalHost().getHostAddress(); en equipos con interfaces virtuales debe revisarse la dirección elegida. Para añadir un nodo se actualizan la lista nodos y sus propiedades, sin cambiar fuentes Java.

4.5. Algoritmo de distribución y balanceo de carga

Solo participan los trabajadores aceptados dentro de la ventana de registro. El maestro cuenta filas y determina L, el máximo de bytes UTF-8 por fila incluyendo LF. Para el nodo i, el espacio utilizable es S_i = min(límite configurado, espacio libre informado). El límite de filas se calcula con divisiones enteras:

$$M_i = \max(0, \text{floor}((\text{floor}(S_i / 3) - 1024) / L))$$

La reserva conservadora destina hasta un tercio a la partición y deja espacio para resultados y temporales. Balanceador.repartir mantiene una cuota por nodo. En cada iteración divide las filas pendientes proporcionalmente a los pesos de los nodos con espacio; trunca al entero inferior y aplica el tope restante. El excedente se vuelve a repartir. Si todas las propuestas son cero, asigna filas de una en una en el orden de los nodos, previamente ordenados por ID. Si no queda capacidad de almacenamiento, lanza IOException.

Tabla 4. Distribución efectiva con capacidades 4, 2 y 1

Nodo	Peso	Asignados	Procesados
nodo1	4	57 144	57 144
nodo2	2	28 571	28 571
nodo3	1	14 285	14 285
Total	7	100 000	100 000

Fuente: evidencias/p2_carga.csv y logs de maestro y trabajadores del ZIP. Los restos explican la diferencia respecto de la proporción exacta.

La prueba de almacenamiento redujo nodo1 a 1 MiB y obtuvo 6 701, 62 200 y 31 099 registros para nodo1, nodo2 y nodo3. Se conservaron las 100 000 filas y el resultado coincidió con el serial. Esto demuestra que un peso alto no prevalece sobre el límite de almacenamiento. No hay medición automática de velocidad del hardware: los pesos son declarados por configuración y no cambian durante el trabajo.

4.6. Implementación serial

El modo Coordinador serial dataset carpeta llama a Procesamiento.procesar sobre el CSV completo en un único proceso. BufferedReader entrega una línea por vez; se validan los seis campos y se rechazan IDs repetidos o desordenados. Cada importe se convierte con BigDecimal a un long en céntimos y se exige conversión exacta. BigDecimal permite expresar decimales con precisión arbitraria antes de esta conversión (Oracle, s. f.-a).

Listado 8. Conversión monetaria y agregación por cuenta

```
long monto=new BigDecimal(c[4])
    .movePointRight(2).longValueExact();
if(monto<=0) throw new IOException("Monto no positivo");
r.agregar(c[1],new Cuenta(1,
    c[3].equals("DEPOSITO")?monto:0,
    c[3].equals("RETIRO")?monto:0));
```

Fuente: Procesamiento.java, procesar(Path). Saltos de línea ajustados.

Resumen utiliza TreeMap<String, Cuenta>. Se acumulan cantidad, depósitos y retiros con Math.addExact, y la variación se calcula por resta exacta. guardar escribe cuentas ordenadas y una fila TOTAL. El consumo de memoria es proporcional a las cuentas distintas; no es constante ni permite un número ilimitado de cuentas. El dataset completo no se carga en RAM.

4.7. Implementación paralela distribuida

El maestro se inicia antes de los trabajadores y acepta sus registros BANCO1, ID, IP y espacio. Escribe particiones contiguas con cabecera y líneas completas, por lo que una transacción no se corta entre archivos. Para cada conexión, una tarea envía UUID, cuota y CSV; el trabajador verifica y procesa su partición en su propia JVM.

Al retornar el parcial, el maestro valida el conteo comunicado y el reconstruido desde sus cuentas. Resumen.unir suma los agregados por clave: una cuenta presente en varias particiones se consolida correctamente. Future.get espera cada resultado y propaga fallos. Solo si la cantidad global coincide con la inspección se genera EXITO.txt. No hay reintentos automáticos; las particiones de un trabajo exitoso se asignan una sola vez.

4.8. Resultados y rendimiento

La revalidación inició cuatro procesos Java independientes en una misma computadora: un coordinador y tres trabajadores conectados a 127.0.0.1:5000. El modo serial y el distribuido procesaron el mismo dataset de 100 000 registros y 1 000 cuentas. La comparación confirmó igualdad SHA-256 de los CSV finales completos.

Tabla 5. Resultados de la revalidación de la pregunta 2

Indicador	Serial	Distribuido
Registros procesados	100 000	100 000
Tiempo de ejecución	522,349 ms	2 028,945 ms
Participantes	1 proceso de cálculo	1 maestro y 3 trabajadores
Resultado	serial.csv	consolidado.csv equivalente

Fuente: revalidacion/p2_demo.log. El tiempo distribuido incluye inspección, espera, particionamiento, comunicación, agregación y escritura. El serial incluye lectura, agregación y escritura.

Speedup = 522,349 / 2 028,945 = 0,2574. La ejecución distribuida tardó aproximadamente 3,88 veces el tiempo serial. El registro anterior del ZIP reporta 456,756 ms y 1 404,103 ms, respectivamente, con speedup 0,3253. Ambas mediciones muestran sobrecarga para este experimento local; no permiten extrapolar tiempos a varias computadoras.

Tabla 6. Consolidación monetaria de las 100 000 transacciones

Magnitud global	Resultado en céntimos
Depósitos	12 619 058 587
Retiros	12 420 351 120
Variación neta	198 707 467

Fuente: evidencias/p2_resumen.csv del ZIP y revalidacion/p2_serial.csv. Variación neta = depósitos − retiros; no se infiere un saldo inicial.

4.9. Análisis de la pregunta 2

El reparto heterogéneo y la consolidación funcionaron, pero los pesos no hacen que procesos locales ejecutados en el mismo hardware tengan potencias físicas distintas. La lectura, partición y consolidación centralizadas, las transferencias y los hashes añaden coste. En una LAN también influirán ancho de banda y latencia. Falta medir equipos reales, repetir ensayos y registrar recursos antes de concluir escalabilidad o aceleración.

5. Pruebas y validación

Se distinguieron dos conjuntos de evidencia: los registros incluidos en Examen_Parcial_Java.zip y una revalidación posterior de los fuentes extraídos de ese archivo. Se comprobó el manifiesto SHA256SUMS.txt del ZIP. Las nuevas compilaciones usaron JDK 25.0.1 y --release 17; no se modificó el código Java para redactar el informe.

Tabla 7. Matriz de validación de la pregunta 1

Código	Prueba	Esperado	Obtenido	Estado
P1-01	Compilar fuentes extraídos	Compilación sin errores	Salida 0	APROBADO
P1-02	Serial y paralelo A–H	8 tareas completas	8 INICIO y 8 FIN por modo	APROBADO
P1-03	Precedencias del grafo	Inicio posterior al FIN de dependencias	CSV verificados	APROBADO
P1-04	Archivos de ambos modos	A.dat–H.dat equivalentes	SHA-256 A–H OK	APROBADO
P1-05	ERROR entre bloques	2 apariciones	2 apariciones; salida 0	APROBADO
P1-06	Entrada ausente	Error; sin EXITO.txt	Fallo esperado registrado	APROBADO
P1-07	Concurrencia efectiva	Solapamiento de tareas independientes	A/B y D/E en registros	APROBADO

Fuentes: revalidacion/Pregunta1_compilacion.log, p1_comparar.log, p1_*_registro.csv, p1_patrones.log; evidencia original p1_entrada_ausente.log y registros del ZIP.

La ausencia de una entrada produce una excepción esperada y código de salida 1: el caso de prueba se aprueba porque el programa rechaza la ejecución incompleta. APROBADO no significa que una operación inválida haya terminado exitosamente. Las comparaciones de precedencias se hicieron con System.nanoTime dentro de una misma JVM; no se utilizaron relojes de equipos distintos.

5.1. Validación del procesamiento distribuido

Tabla 8. Matriz de validación de la pregunta 2

Código	Prueba	Esperado	Obtenido	Estado
P2-01	Compilar fuentes extraídos	Sin errores	Salida 0	APROBADO
P2-02	Tres JVM trabajadoras	Transferir y procesar particiones	100 000/100 000 filas	APROBADO
P2-03	Pesos 4:2:1	Reparto desigual	57 144 / 28 571 / 14 285	APROBADO
P2-04	Límite de nodo1 a 1 MiB	Redistribuir excedente	6 701 / 62 200 / 31 099	APROBADO
P2-05	Serial frente a consolidado	CSV equivalentes	SHA-256 OK	APROBADO
P2-06	Referencia Python Decimal	Mismos conteos e importes por cuenta	Coincidencia original registrada	APROBADO
P2-07	Ningún trabajador	Rechazar trabajo	No hay trabajadores disponibles	APROBADO
P2-08	Solo nodo1 disponible	Procesar con el conectado	Salida igual al serial	APROBADO
P2-09	Puerto, IP o ID inválidos	Rechazar configuración	Errores esperados	APROBADO
P2-10	Dos filas y tres trabajadores	Admitir cuota cero	Mismo resultado serial	APROBADO
P2-11	IDs duplicados	Rechazar dataset	Error registrado	APROBADO
P2-12	Archivo truncado o hash erróneo	No publicar archivo	PruebaTransferencia OK	APROBADO
P2-13	Desconexión tras registro	Abortar sin éxito global	Sin EXITO.txt	APROBADO
P2-14	Balanceador sin espacio total	Rechazar asignación	IOException esperada	APROBADO
P2-15	Varias laptops y firewall LAN	Transferencia entre equipos físicos	Sin ejecución física	NO EJECUTADO
P2-16	Scripts en Linux	Compilar y ejecutar	Preparados con LF; sin Linux	NO EJECUTADO

Fuentes: evidencias/PRUEBAS.md, p2_* del ZIP y registros de revalidación. El bloqueo inicial del sandbox para sockets se superó con permiso de TCP local; no fue un defecto del programa.

5.2. Alcance de las verificaciones

Se reejecutaron la compilación de ambas preguntas, la comparación serial/paralela de P1 con entradas de 16 MiB, el control de dependencias y hashes, PruebaPatrones, PruebaBalanceador, PruebaTransferencia y la demostración TCP de P2 con 100 000 registros. Los escenarios de error adicionales de la matriz se respaldan en las pruebas y logs originales incluidos en el ZIP; no todos se repitieron durante la elaboración del informe.

Los archivos disponibles son registros de consola y CSV, no capturas de pantalla de terminales. Los extractos del documento se presentan como texto y los diagramas explican el código; ninguno se presenta como una captura real. Los mensajes de error de casos negativos se conservaron como evidencia de detección.

El primer intento de prueba del clúster en el entorno restringido produjo Permission denied: getsockopt. Al habilitar la ejecución TCP local, las cuatro JVM completaron el trabajo. El fallo inicial corresponde al permiso del entorno de prueba y se diferencia de los errores intencionales del programa, como la falta de trabajadores o los IDs duplicados.

La implementación no garantiza operación ante nodos maliciosos, cambios concurrentes del dataset ni fallos de energía. SHA-256 detecta corrupción de los bytes transferidos, pero no autentica al emisor. La validación de IDs y cuotas evita omisiones o duplicaciones en los trabajos exitosos bajo las condiciones de entrada estable y nodos confiables usadas en el experimento.

El resultado de P1 está limitado a manipulación de archivos y conteo de un patrón. La búsqueda estadística de tendencias o correlaciones del enunciado impreso no forma parte del código. P2 implementa balanceo previo por pesos, no balanceo adaptativo basado en la carga instantánea de las computadoras. Estas diferencias delimitan la cobertura lograda sin atribuir funciones inexistentes al proyecto.

6. Comparación entre las dos preguntas

Tabla 9. Comparación de las implementaciones desarrolladas

Aspecto	Pregunta 1	Pregunta 2
Unidad de trabajo	Operaciones diferentes A–H	Particiones de transacciones
Ejecución paralela	Hilos dentro de una JVM	Maestro y trabajadores en JVM distintas
Dependencias	Grafo explícito con dos ramas	Inspección, partición, cálculo y consolidación
Distribución de carga	Determinada por las tareas del grafo	Cuotas por peso y almacenamiento
Recursos compartidos	Disco y registro; archivos separados por tarea	Disco del maestro y conexión por trabajador
Comunicación	Archivos y estado de CompletableFuture	TCP con longitud, hash y bytes
Sincronización	allOf, thenRunAsync y join	Sockets, Future.get y validación de cuotas
Corrección	SHA-256 de A.dat–H.dat	Conteos y SHA-256 del CSV agregado
Ventaja demostrada	Solapamiento respetando dependencias	Reparto heterogéneo y agregación correcta
Limitación observada	Rendimiento variable en el mismo disco	Sobrecoste local y maestro centralizado

En P1, la dependencia entre resultados impone cuándo una operación puede avanzar. En P2, la independencia de las filas permite aplicar el mismo algoritmo a particiones separadas y sumar agregados. La presencia de cuentas repetidas entre nodos no exige ordenar las transacciones por cuenta: Resumen.unir consolida las claves comunes.

Las dos soluciones leen y escriben datos persistidos, pero el intercambio de P1 permanece dentro de un sistema de archivos local. P2 transfiere realmente archivos entre procesos y admite direcciones LAN configurables; su capacidad de operar en varias computadoras está implementada, aunque la evidencia experimental disponible corresponde a una sola computadora.

7. Conclusiones

La pregunta 1 implementó las ocho operaciones físicas A–H mediante RandomAccessFile. El procesamiento por bloques y el uso de seek permitieron copiar, replicar, transformar y extraer contenido sin cargar los archivos completos en memoria. La búsqueda KMP conservó patrones que atraviesan el límite de un bloque.

La sincronización con CompletableFuture reprodujo el grafo del examen. Los registros verificaron precedencias y ejecución superpuesta de tareas independientes. La igualdad SHA-256 de los ocho archivos confirma que la ejecución concurrente preservó el resultado de la serial en los experimentos realizados.

La pregunta 2 distribuyó trabajo efectivo por TCP entre un maestro y tres procesos trabajadores. El reparto de 57 144, 28 571 y 14 285 filas evidencia la influencia de los pesos 4:2:1. Al restringir el almacenamiento del primer nodo, el excedente se reasignó y se conservaron las 100 000 transacciones.

La agregación en céntimos enteros y la consolidación por cuenta produjeron los mismos resultados seriales y distribuidos. Los controles de cuotas, integridad de transferencia y marcadores de éxito permitieron detectar trabajos incompletos; la variación neta obtenida no constituye un saldo bancario sin datos iniciales.

La corrección no implicó una aceleración estable. En la revalidación, los speedups fueron 0,179 para P1 y 0,2574 para P2. La variación frente a registros anteriores y el uso de un único equipo impiden concluir ventajas generales de rendimiento. La evaluación entre computadoras físicas continúa pendiente.

8. Recomendaciones

Para evaluar rendimiento, se recomienda repetir cada escenario varias veces con datasets de distintos tamaños, alternar el orden de ejecución y separar calentamiento de medición. Deben registrarse CPU, memoria, tipo de almacenamiento, actividad concurrente y condiciones de caché. Los resultados deberían reportarse con una medida de dispersión, además del tiempo central; estas prácticas no están automatizadas en el proyecto actual.

Para P1, conviene medir por separado el tiempo de las operaciones de disco, el registro en consola y el conteo del patrón. Probar archivos ubicados en unidades distintas permitiría estudiar la contención sin alterar el grafo. Si se requiere cumplir el alcance estadístico completo del enunciado, debe definirse e implementarse un análisis adicional de tendencias o correlaciones, que hoy no está desarrollado.

Para P2, debe ejecutarse una prueba LAN con maestro y trabajadores en equipos físicos diferentes, capacidades documentadas y permisos de firewall controlados. Se recomienda comparar pesos declarados con rendimientos observados y evaluar un reparto adaptativo o por lotes. La versión actual asigna una partición estática por trabajador y no redistribuye tareas después del inicio.

Antes de usar redes no confiables, deberían añadirse autenticación y cifrado, además de límites más estrictos de protocolo y validación de los parciales. Para recuperación operativa, conviene incorporar identificadores persistentes, reintentos idempotentes y un registro de trabajos que permita distinguir resultados confirmados de intentos interrumpidos. Estas son mejoras propuestas, no funciones existentes.

Cuando aumente la cantidad de cuentas distintas, se recomienda evaluar agregación externa o particionamiento por cuenta para limitar el mapa en memoria. También conviene vigilar el disco temporal del maestro, definir limpieza de UUID antiguos y proteger la entrada contra modificaciones durante el procesamiento. La reserva de espacio por trabajador es conservadora, pero no sustituye una gestión integral del almacenamiento.

9. Referencias

Examen_Parcial_Java [Código fuente, configuración y registros de prueba]. (2026). Archivo Examen_Parcial_Java.zip entregado con el desarrollo del examen parcial.

López V., P. E. (2026, 15 de octubre). *Examen parcial de Programación Concurrente y Paralela* [Evaluación del curso]. Universidad Nacional Mayor de San Marcos. Archivo recibido: EP - PCP.docx.

López V., P. E. (s. f.-a). *ClusterComputing_1.java* [Código Java de referencia del curso]. Universidad Nacional Mayor de San Marcos.

López V., P. E. (s. f.-b). *Configuracion.txt* [Archivo de asignación de trabajo por IP]. Universidad Nacional Mayor de San Marcos.

Oracle. (s. f.-a). *BigDecimal*. En Java Platform, Standard Edition 17 API specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html

Oracle. (s. f.-b). *CompletableFuture*. En Java Platform, Standard Edition 17 API specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/CompletableFuture.html

Oracle. (s. f.-c). *RandomAccessFile*. En Java Platform, Standard Edition 17 API specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/RandomAccessFile.html

Oracle. (s. f.-d). *Socket*. En Java Platform, Standard Edition 17 API specification. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/Socket.html

Prompt para agente de programación sobre el examen parcial de Programación Concurrente y Paralela. (2026). [Texto pegado con las instrucciones de desarrollo y precisiones atribuidas al docente, proporcionado junto con los materiales del proyecto].

10. Anexos

10.1. Estructura del proyecto entregado

```
Examen_Parcial_Java/
  Pregunta1/
    src/          Principal, Serial, Paralelo, Operaciones,
                  PruebaPatrones
    entrada/      registro1.txt, registro2.txt
    scripts/      compilar y ejecutar (.bat y .sh)
    salida/       se genera durante la ejecución
    README.md
  Pregunta2/
    src/          Coordinador, Trabajador, Configuracion,
                  Procesamiento, Transferencia, Balanceador,
                  Generador y clases de prueba
    config/       cluster.properties, nodo1-ip.txt,
                  nodo2-ip.txt, nodo3-ip.txt
    dataset/      muestra.csv
    resultados/   se genera durante la ejecución
    scripts/      compilar, ejecutar, demo_local
    README.md
  evidencias/     logs, cargas, resumen y matriz de pruebas
  README.md
  SHA256SUMS.txt
```

El ZIP excluye archivos .class, datasets grandes y directorios de trabajo generados. Los scripts crean build y las aplicaciones generan sus carpetas de salida. La revalidación del informe se realizó en una copia extraída, conservando el ZIP de entrega original.

10.2. Identificación de la fuente analizada

Archivo: Examen_Parcial_Java.zip. Tamaño: 63 141 bytes. Contiene 81 entradas, incluidas carpetas. Se verificaron la integridad del archivo ZIP y los hashes del manifiesto interno antes del análisis.

```
SHA-256 del ZIP:
49ea4810d10bf57acbc0d7ad788111fb97bcb1eb43ed8c5a78ef825da2128589
```

El examen disponible se llama EP - PCP.docx y corresponde al material mencionado como DOC-20261015-WA0036.docx en las indicaciones iniciales. Se revisaron su texto y el diagrama de dependencias, junto con los dos archivos docentes de referencia.

10.3. Clases y métodos principales

Tabla 10. Mapa de clases de las dos aplicaciones

Archivo	Responsabilidad y métodos relevantes
Principal.java	Orquestación P1: nodo, ejecutar, anotar, generar y main.
Serial.java / Paralelo.java	Entradas independientes que seleccionan el modo de P1.
Operaciones.java	segmento, unir, extraer, transformar, patrones y sha.
Coordinador.java	Conexion, inspeccionar, distribuir y modos serial/maestro/comparar/ip.
Trabajador.java	Registro del nodo, recepción, procesamiento y retorno en main.
Configuracion.java	Carga de Properties; texto, numero, puerto, ip, leerIp y local.
Generador.java	Creación secuencial y reproducible del CSV en main.
Balanceador.java	repartir: cuotas por peso limitadas por filas máximas.
Procesamiento.java	Cuenta.sumar; Resumen.agregar/unir/cantidad/guardar; procesar y leerResumen.
Transferencia.java	sha, enviar y recibir con tamaño, hash y temporal.
Clases de prueba	PruebaPatrones, PruebaBalanceador y PruebaTransferencia.

10.4. Ejemplos de salida verificados

Listado 9. Cabecera y total del resumen bancario de 100 000 registros

```
cuenta,cantidad,depositos_centimos,retiros_centimos,variacion_neta_centimos
TOTAL,100000,12619058587,12420351120,198707467
```

El campo variacion_neta_centimos se obtiene como depósitos menos retiros. Su valor global equivale a 1 987 074,67 unidades monetarias del dataset; no se asignó una moneda ni un saldo inicial que no figuren en la entrada.

10.5. Comandos para reproducir la pregunta 1

Requisito: JDK 17 o posterior con java y javac disponibles en PATH o JAVA_HOME. Desde la carpeta Pregunta1 en Windows:

```
scripts\compilar.bat
scripts\ejecutar.bat generar 16777216
scripts\ejecutar.bat serial
scripts\ejecutar.bat paralelo
scripts\ejecutar.bat comparar
java -cp build PruebaPatrones
```

En Linux, usar sh scripts/compilar.sh y sh scripts/ejecutar.sh con los mismos argumentos. comparar vuelve a ejecutar ambos modos y verifica los hashes A–H; por ello sus tiempos constituyen una nueva medición, no reutilizan necesariamente los de ejecuciones individuales anteriores.

10.6. Comandos para reproducir la pregunta 2

Desde Pregunta2 en Windows, el lanzador compila, genera los datos, ejecuta serial, inicia cuatro JVM y compara. Python 3 solo es necesario para ese lanzador:

```
scripts\demo_local.bat 100000
```

Alternativamente, compilar y generar antes de abrir cuatro terminales:

```
scripts\compilar.bat
java -cp build Generador dataset/transacciones.csv 100000 1000
java -cp build Coordinador serial dataset/transacciones.csv resultados
java -cp build PruebaBalanceador
java -cp build PruebaTransferencia
```

En Linux: sh scripts/demo_local.sh 100000. Los scripts .sh están preparados con saltos LF, pero su ejecución en Linux quedó pendiente de validación.

10.7. Inicio manual de maestro y trabajadores

Iniciar primero el maestro y luego cada trabajador en una terminal distinta, todas ubicadas en Pregunta2. No esperar a que el maestro termine para abrir los trabajadores. Ampliar espera.ms si se necesitan más de 10 segundos para iniciarlos manualmente.

```
java -cp build Coordinador maestro config/cluster.properties \
  dataset/transacciones.csv resultados

java -cp build Trabajador config/cluster.properties nodo1 \
  config/nodo1-ip.txt trabajo/nodo1
java -cp build Trabajador config/cluster.properties nodo2 \
  config/nodo2-ip.txt trabajo/nodo2
java -cp build Trabajador config/cluster.properties nodo3 \
  config/nodo3-ip.txt trabajo/nodo3

java -cp build Coordinador comparar resultados
```

Los saltos con barra inversa muestran continuación para una terminal Linux. En Windows escribir cada comando en una sola línea, sin las barras de continuación.

10.8. Preparación de la demostración en una LAN

Copiar Pregunta2 y compilar en cada laptop. Consultar IPv4 con ipconfig en Windows o ip addr en Linux; seleccionar una IP real del maestro y establecerla en todas las copias de cluster.properties. Configurar una IP e ID propios por trabajador y guardar esa IP en nodoN-ip.txt. La dirección 192.168.1.100 puede usarse solo como ejemplo de configuración; no corresponde a una prueba realizada.

Permitir TCP entrante al puerto configurado del maestro desde la red privada y la salida de los trabajadores. No se requieren carpetas compartidas ni puertos de escucha de trabajadores. Verificar que el Wi-Fi no aísle a sus clientes. Ejecutar los comandos anteriores y conservar logs de cada equipo. La prueba física LAN y la validación de firewall están pendientes.

10.9. Evidencias de comparación y trazabilidad

Tabla 11. Fuentes de evidencia usadas para el informe

Evidencia	Contenido verificado
evidencias/p1_comparacion.log	Medición original: 2 875,833 ms seriales, 2 110,523 ms paralelos, SHA A–H OK.
evidencias/p1_*_registro.csv	Eventos por nodo, hilos y marcas monotónicas.
evidencias/p2_comparacion.log	Medición original: 456,756 ms seriales, 1 404,103 ms distribuidos y SHA OK.
evidencias/p2_carga.csv	Pesos 4:2:1 y cuotas de 100 000 registros.
evidencias/p2_carga_limite.csv	Redistribución al limitar nodo1 a 1 MiB.
evidencias/p2_resumen.csv	Agregados por 1 000 cuentas y total monetario.
evidencias/PRUEBAS.md	Relación de verificaciones y restricciones de las pruebas originales.
revalidacion/p1_comparar.log	Nueva medición: 712,352 ms y 3 976,087 ms; SHA A–H OK.
revalidacion/p2_demo.log	Nueva medición: 522,349 ms y 2 028,945 ms; 100 000 filas y SHA OK.

Las rutas revalidacion son relativas a informe_trabajo, carpeta de trabajo del informe; las otras rutas pertenecen al proyecto extraído del ZIP.

El hash del resumen bancario original es fc5b4d14b90e67427a9c501582197f6be029f37341c23ffc56243044a8a60547. La revalidación generó el mismo contenido. Los registros UTC de las ejecuciones originales del 16 de octubre a las 00:39 corresponden al 15 de octubre por la noche en Lima (UTC−5).